上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

SJTU



互联网产品开发与运维

任锐

2026/10/20

饮 水 思 源 • 爱 国 荣 校

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

资源管理：分配与隔离

模块 6 · 资源管理（1/3）

饮 水 思 源 • 爱 国 荣 校

SJTU

本讲位置：从「账本」到「分配」

上一讲 · L-08

三层指标、SLO 与错误预算：模块 5 把软件从「架构图」变成了「账本」

遗留问题：账本有了，每个服务能拿多少资源，由谁决定?

本讲问题

一台机器上的几个服务，如何分配资源才能既不互相拖累，也不白白浪费?

引例（设定）

开放日前夜，4 个 Tomcat 迁入容器，与「选课结果统计」批处理混部。9:00 节点 CPU 不到 70%，提交 p99 却每隔几秒超过 1 s（**症状 A：变慢**）；9:04 一个 Tomcat 被 OOMKilled（137），节点内存还剩三成（**症状 B：被杀**）

本讲结构

1 预备知识：资源管理管什么

2 隔离：阀门怎么拧

3 分配：给多少

4 课程实践：选课平台的配额表

课程实践以一张「配额表」回答引例

本模块 资源管理（L-09 ~ L-11）：隔离与分配（本讲）· 调度（L-10）· 伸缩（L-11）

来源：L-08 第 33 页；L-06 第 27 页；引例中的数字均为设定值

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

一、预备知识

第九讲 · 分配与隔离

饮水思源 • 爱国荣校

SJTU

预备知识 • 地图

资源管理的地图：三问加一个保证

给多少	放哪儿	何时变
分配：配额与超卖	调度：装箱与反亲和	伸缩：定时与反应式
本讲	L-10	L-11

隔离：保证给的就是拿到的

cgroup 的阀门；本讲

判断标准 这份资源给谁、给多少？不够或超了，谁受损、代价是什么？

三问加一个保证

分配定数量，调度定位置，伸缩定时间；隔离让分配的数量真正生效

L-08 第 33 页列出「调度、扩缩容、隔离」；本模块按「隔离与分配 → 调度 → 伸缩」的顺序讲

先有阀门，分配才有意义；先定数量，才谈得上放置和增减

要点 先有阀门（隔离），才谈得上分配；先分配，才谈得上放置和增减

来源：L-08 第 33 页

资源的性质：变慢，还是失败

资源	类型	超出时	症状与出处	引例
CPU	可压缩	变慢：排队、被节流	throttled 计数（L-06 第 27 页）	**症状 A**
网络带宽	可压缩	限速排队、重传	重传率、RTT（L-07 第 19、21 页）	
磁盘 IOPS	可压缩	排队等待	await 上升（L-07 第 16 页）	
内存	不可压缩	失败：被杀（OOM，137）	OOMKilled（L-06 第 7、27 页）	**症状 B**
fd、进程数、端口	不可压缩	报错、拒绝	Too many open files（L-06 第 8 页）	
磁盘空间	不可压缩	写入失败、宕机	思考题 1 第四幕	

要点 可压缩资源超出时变慢，不可压缩资源超出时失败；所以超卖 CPU 与超卖内存是两回事（第 21 页）

来源：L-06、L-07 讲义；思考题 1

负载的性质：谁让谁

类型	选课平台中的例子	延迟敏感	可中断、可重跑	迁移代价	管理含义
在线无状态	Tomcat	高	否	低（L-05 第 25 页）	**保证资源**
离线批处理	「选课结果统计」	低	是	低	**可被挤压、可被终止**
有状态	MySQL、Redis	高	否	高：数据随实例	**独占、少迁移**

批处理「选课结果统计」（本讲设定）

开放日 8:30 ~ 10:00 运行，统计各课程的选课结果，供教务导出

与 Q-2 三年后阶段的「离线推荐」无关，本讲只用这一个批处理

资源紧张时，谁让?

在线服务的延迟直接被学生感知；批处理晚一些完成，代价是统计结果推迟

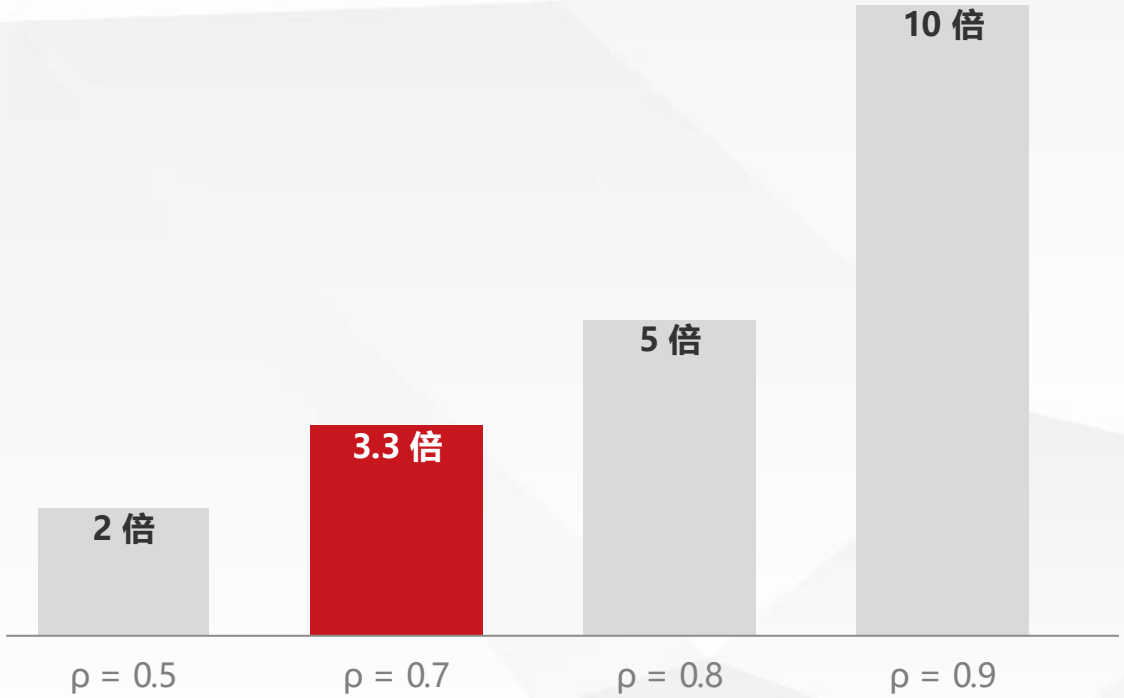
有状态服务换机器要搬数据，宁可独占，也不要和别人抢

要点 资源紧张时必须有一方让步，负载类型决定谁让

来源：L-05 第 24、25 页；批处理为本讲设定

目标冲突：利用率、延迟、成本

平均响应时间 ÷ 服务时间 = 1 ÷ (1 − ρ)（L-07 第 6 页）



ρ = 0.7 即 L-07 的容量线：上限 × 0.7；越过之后，等待时间增长得越来越快

三个目标互相牵制

利用率高：机器少，成本低

延迟低：需要留余量，利用率就上不去

共享：提高利用率，也带来邻居干扰

约束来自 L-08

SLO 与错误预算决定余量留多少、干扰能容忍多少

开放日前 10 分钟：提交 p99 ≤ 1 s（L-08 第 28 页）

要点 资源管理是在 SLO 约束下提高利用率，而不是单纯追求高利用率

来源：L-07 第 6 页（M/M/1 排队模型）；L-08 第 19、21、28 页

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

二、隔离

第九讲 · 分配与隔离

饮水思源 • 爱国荣校

SJTU

cgroup v2 的阀门

控制器	管什么	关键文件（cgroup v2）	对应症状
cpu	CPU 时间	cpu.weight：份额，默认 100，范围 1～10000 cpu.max：配额，格式「$MAX $PERIOD」，默认 max 100000 cpu.stat：nr_periods、nr_throttled、throttled_usec	**节流：症状 A**
memory	内存（含页缓存）	memory.high：软限，超出后被限速并回收，不触发 OOM memory.max：硬限，无法回收时触发 OOM killer memory.events：max、oom、oom_kill 等计数	**OOMKilled：症状 B**
io	块设备 IO	io.weight、io.max、io.stat	本讲不展开
pids	进程与线程数	pids.max	本讲不展开

回顾 L-06 第 27 页 容器是受限制的进程：namespace 决定它能看见什么，cgroup 决定它能用多少

例：systemd-run -p CPUQuota=300% … 即写入 cpu.max = 300000 100000，每 100 ms 周期最多使用 300 ms CPU 时间

要点 每个阀门对应一种症状：看症状，就知道是哪个阀门到了上限

来源：Linux kernel docs, Control Group v2（2026-10-14 查阅）

份额与配额：两种分法

争用时：份额 A : B = 300 : 100



B 空闲时：只有份额，A 可用满 8 核



A 加配额 4 核：B 空闲也不放行



节点 8 核，每格 1 核；A、B 均为 CPU 密集

问题	原因	对策
只用硬配额	空闲时也不能借用，造成浪费；突发时被节流	保证用份额；只在需要封顶时设配额
只用份额	只在争用时生效，平时看不出差别，开放日才暴露	压测或演练时人为制造争用，看各方实际拿到多少

要点 份额决定争用时谁多拿，配额决定最多拿多少

来源：Linux kernel docs, Control Group v2（cpu.weight、cpu.max）；示意图为算术推演

CFS 配额算术

配额算术

周期 100 ms（默认）；limit L 核 → 每个周期 L × 100 ms 的 CPU 时间

n 个线程同时忙：配额在 L × 100 ÷ n ms 内用完，此后停到周期结束

忙线程数 n	每周期运行	每周期被停
3	100 ms	0
6	50 ms	50 ms
12（引例）	**25 ms**	**75 ms**
24	12.5 ms	87.5 ms

L = 3 核，配额 300 ms。前提：n 个线程同时在跑、核数足够；等待 IO 的线程不算忙

一个 100 ms 周期内，每个线程的时间线



现场演示 在 3 核配额的 cgroup 中运行纯计算程序，n = 3、6、12 各 4 秒

读 cpu.stat 的 nr_periods、nr_throttled、throttled_usec，与左表对照

实测值来自示例程序，课上给出；它不等于场景中的 Tomcat，不替换第 25 页的设定值

要点 节流取决于周期内的瞬时并发，而非平均 CPU：平均用量低于 limit 也会被停，停顿直接进入 p99

来源：Linux kernel docs, CFS Bandwidth Control（cpu.cfs_period_us 默认 100 ms）；表为算术推演

节流怎么发现，怎么治

问题	原因	对策
节点 CPU 不满，p99 却抖动	周期内配额用完，线程被停到周期结束	读 cpu.stat：nr_throttled ÷ nr_periods limit 按峰值 ÷ 0.7 定；或去掉硬 limit 只留份额；或让并发线程数不超过 limit 核数
运行时按宿主机核数创建线程，忙线程数远超 limit	旧版本读不到 cgroup 的 CPU 限制	用能识别容器限制的版本，或显式设置：JDK 10 起（8u191 回移）；cgroup v2 需 JDK 15（11.0.16、8u372 回移）；Go 1.25 起 GOMAXPROCS 按 CPU limit 取值

cpu.stat 的三个计数

nr_periods：经历的周期数；nr_throttled：被节流的周期数

throttled_usec：被停的累计时间，按 CPU 分别累计，可超过墙钟时间

回顾 L-06 第 27 页

「检查 throttled 计数；按峰值设定配额，或取消硬性配额」

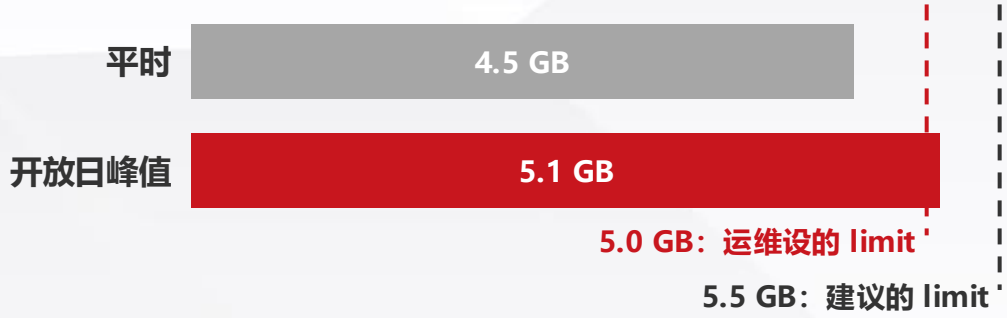
实验 3 会在自己的主机上复现并测量这一现象

要点 节流有计数器：看到节流比例，就能判断是不是它

来源：Linux kernel docs, Control Group v2；OpenJDK JDK-8146115、JDK-8230305；Go 1.25 Release Notes

内存：limit 怎么定

一个 Tomcat 的 RSS（GB，设定）



构成	堆	堆外	线程栈	元空间等	连接缓冲	峰值 RSS
GB	4.0	0.5	0.2	0.2	0.2	**5.1**

limit = 峰值 5.1 + 余量约 8% ≈ 5.5 GB

构成即 L-06 第 7 页的内存账：堆 + 堆外 + 线程栈 + 元空间等 + 连接缓冲 + 余量

两个阀门

memory.high：超出后被限速并回收，变慢但不触发 OOM

memory.max：超出且无法回收时触发 OOM killer，进程退出码 137

页缓存

也计入容器内存，但内存紧张时先被回收，通常不是 OOM 的主因（L-07 第 11 页）

节点内存够，容器仍被杀 超出的是 cgroup limit，而非整机内存 → limit 按观测峰值加余量定，不按平时值定

要点 内存没有「慢一点」，只有「够」和「被杀」

来源：Linux kernel docs, Control Group v2（memory.high、memory.max）；L-06 第 7 页；数字为本讲设定

隔离的盲区与隔离强度

共享资源	cgroup 为何管不到	症状	对策
末级缓存、内存带宽	硬件共享，cgroup 不计量	邻居一忙，自己就变慢	专用节点、绑核
磁盘队列与云盘限额	限额按整块云盘计，同盘进程共用；io.max 只能限上限，不能保证	await 上升	有状态服务独占
连接跟踪表、PPS（L-07 第 20 页）	节点级共享	偶发超时、丢包	查内核日志、换规格
内核锁、页缓存争用	内核内部	难以归因	减少混部

隔离强度

进程 → 容器 → 虚拟机 → 裸机：隔离越强，密度越低、成本越高（L-06 第 26 页）

选课平台的选择

Tomcat 与批处理：容器加 cgroup 够用，密度高
MySQL：有状态、对 fsync 敏感、盲区多，独占节点

要点 没有阀门的地方，就是邻居干扰的通道；隔离强度按负载类型选，不是越强越好

来源：L-06 第 26 页；L-07 第 20 页；B. Gregg, Systems Performance, 2nd ed., Ch. 11

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

三、分配

第九讲 · 分配与隔离

饮水思源 • 爱国荣校

SJTU

分配 · 两个数

request 与 limit

	含义与用途	CPU	内存
request	保证量：调度时按它占位（Σrequest ≤ 节点容量）	cpu.weight：争用时按 request 的比例分；可以用得更多	不设硬限，用于调度和驱逐排序；用量超出 request 时，节点内存紧张便可能被驱逐
limit	上限：阀门	cpu.max：超出即被节流	memory.max：超出且无法回收即被杀

只写 limit 时

Kubernetes 把 request 设为与 limit 相同；两者都不写，就既无保证也无上限

通用词汇

request、limit 是 Kubernetes 的叫法；其他平台叫法不同，落到 cgroup 都是这几个阀门

cgroup v2 上，运行时还可能参考内存 request 设置 memory.min、memory.low

要点 request 管「不被挤」，limit 管「不拖累别人」

来源：Kubernetes docs (v1.37), Resource Management for Pods and Containers（2026-10-14 查阅）

三档 QoS 与压力下的顺序

Guaranteed
每个容器的 CPU、内存 request = limit

最后让

Burstable
设了 request 或 limit，但不满足 Guaranteed

BestEffort
一个 request、limit 都没设

最先让

节点内存紧张时，从下往上让步

压力下的两条路径

kubelet 驱逐：先看用量是否超过 request，再看优先级，再看超出多少；不直接按 QoS 排序，效果上近似左图的顺序

内核 OOM：按 QoS 设 oom_score_adj，Guaranteed −997，BestEffort 1000，Burstable 介于两者之间

对策

在线核心服务：内存 request = limit

批处理：很小的 request 加明确的 limit（Burstable），不用 BestEffort：它没有 limit，失控时无界占用

要点 QoS 级别是「出事时谁先让」的书面答案

来源：Kubernetes docs (v1.37), Pod Quality of Service Classes；Node-pressure Eviction（2026-10-14 查阅）

分配 · 取值

request 怎么定

取值方式	后果	代价
按平均用量	平时够用，峰值时被挤、被节流	成本低，风险高
按峰值	稳	平时大量闲置
按高分位用量加余量	折中：开放日取峰值，平时取高分位	需要观测数据（L-08）

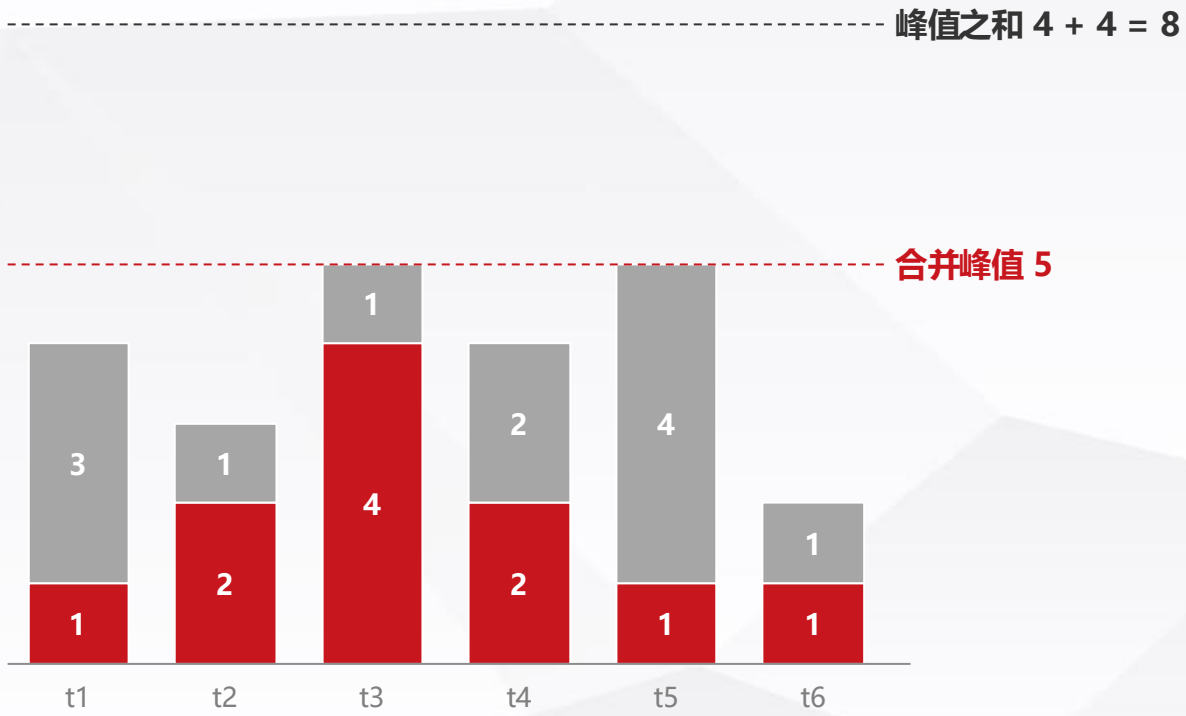
CPU request	≈ 峰值用量：首 10 秒 2.55 核 → 取 2.6 核
CPU limit	≥ 峰值 ÷ 0.7：2.55 ÷ 0.7 = 3.64 → 取 4 核（即 L-07 的 4 核）
内存	request = limit = 观测峰值 5.1 GB 加约 8% 余量 → 取 5.5 GB

要点 用量数据来自 L-08 的指标，余量来自 L-07 的容量线

来源：L-07 第 6、25 页；L-08 第 10 页；数字为本讲设定与推算

超卖（上）：凭什么能——错峰

■ A ■ B 两个服务各时段的 CPU 用量（核，量级示意）



超卖

Σrequest ≤ 容量 < Σlimit：承诺的上限之和超过机器实有量

依据一 · 错峰（统计复用）

左图合并峰值 5，峰值之和 8，省下 3 核；若两者峰值对齐，合并峰值就是 8，没有节省

成立条件：各负载的峰值不同时到

开放日

所有在线服务同时到达峰值，这个依据失效。还能依靠什么？见下页

要点 错峰的前提是「不同时」，开放日恰好是「同时」

来源：用量序列为量级示意，非实测

超卖（下）：优先级兜底，以及内存为什么不行

依据二 · 优先级兜底

CPU 可压缩：争用时压低可重跑的批处理，在线服务不受影响。成立条件：被压的一方可以降级

资源	可压缩	能否超卖	依靠什么	超出的后果
CPU	是	**能**	错峰，或优先级兜底	被压的一方变慢
内存	否	**不超卖：Σlimit ≤ 容量**	—	被杀

内存为什么不能超卖

被压就等于被杀；而且 JVM 堆长大后默认不会及时归还：G1 只在完整 GC 或并发周期后归还，定期归还的选项默认关闭，内存曲线几乎是平的，错峰无从谈起

要点 超卖赌的是「不同时」或「有人可让」；开放日只能靠后者，而且只对 CPU 成立

来源：OpenJDK JEP 346（JDK 12，G1 及时归还空闲内存）

请求级的资源管理

方法	保护什么	代价
限流（令牌桶）	后端不被打穿	超出的请求被拒绝
并发上限（池大小）	线程、连接不被占满（L-06）	排队移到池外
有界队列加超时	不无限排队（L-06 背压）	超时的请求被丢弃
优先级丢弃	过载时保住核心：提交优先于首页查询	低优先级功能降级
舱壁	提交与查询各用独立的线程池、连接池，一个变慢不拖垮另一个	利用率略低

阈值 入口限流 = 容量 × 0.7（L-07）；读的阈值在第 29 页计算

写的阈值由 MySQL 的磁盘决定（L-07：fsync 上限约 250 TPS），而不是 Tomcat；L-11 讨论伸缩时再回到这里

要点 机器级阀门管「谁多拿」，请求级阀门管「放多少进来」

来源：Google SRE Book, Ch. 21 Handling Overload；L-06、L-07 讲义

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

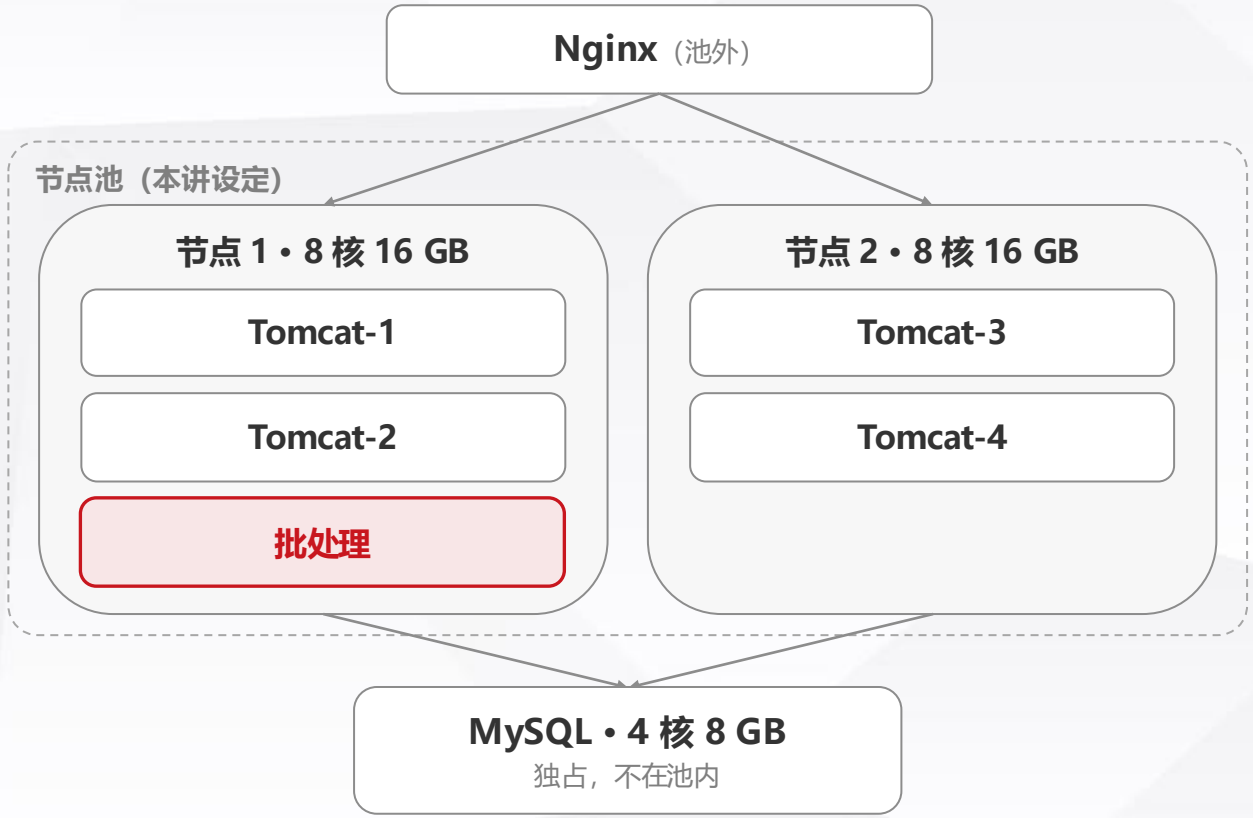
四、课程实践：配额表

第九讲 · 分配与隔离

饮 水 思 源 • 爱 国 荣 校

SJTU

参数设定：L-07 的平台迁入容器



沿用 L-07、L-08

读 2400 QPS、写 300 TPS（前 10 秒 600 TPS）；每请求 CPU 读 3 ms、写 5 ms；开放日前 10 分钟提交 p99 ≤ 1 s

新增（本讲设定）

节点池：2 × 8 核 16 GB，总量与原 4 台 4 核 8 GB 相同

放法：节点 1 放 Tomcat × 2 与批处理，节点 2 放 Tomcat × 2；如何放置是 L-10 的内容

批处理：8:30 ~ 10:00 运行，可重跑、延迟不敏感；平时约 1 核、1.5 GB，全速可用 4 核

Tomcat：CPU 稳态 2.2 核、首 10 秒 2.55 核；RSS 平时 4.5 GB，峰值 5.1 GB；峰值活跃线程约 12

注 数值为设定，与 L-07 的资源账自洽：Tomcat 共需 8.7 核，占 16 核的 54%

来源：L-07 第 24、25 页；L-08 第 28 页

开放日前夜的配额表，与 9:00 的数据

运维的配额表（设定）

负载	CPU	内存
Tomcat × 4	只设 limit：3 核	只设 limit：5.0 GB
批处理	未设置	未设置

运维的考虑：Tomcat 平均只用 2.2 核，3 核已有富余；内存平时 4.5 GB，5.0 GB 够用

9:00 ~ 9:05 的数据（设定）

2.2 / 2.55 核
单实例 CPU：平均 / 首 10 秒

12
活跃线程峰值

38%
nr_throttled ÷ nr_periods

5.1 GB
RSS 峰值

约 67%
节点 1 CPU 利用率

70%
节点 1 内存已用

注 以上均为设定值。第 12 页的演示是示例程序的实测，只能参照量级，不能替换这里的 38%

来源：本讲设定

课堂讨论

1 节流

节点 CPU 不到 70%，Tomcat 的 CPU limit 为 3 核（平均用 2.2 核），为什么提交 p99 仍超过 1 s?

有哪些改法，各有什么代价?

2 被杀

内存 limit 5.0 GB，RSS 峰值 5.1 GB，节点内存还剩三成，为什么仍被杀?

limit 应定多少，依据是什么?

3 批处理

批处理应设为哪个 QoS 级别?

它被挤压或被终止，代价由谁承担?

4 超卖

改为 Tomcat limit 4 核、批处理 limit 4 核后，节点 1 的 Σlimit（CPU）为 12 核，容量 8 核（1.5 倍）。

开放日 9:00 成立吗？超卖在什么条件下安全?

安排 每题小组讨论 2 分钟、汇报 1 分钟，其余时间讲评

讨论要点（1） （2）

1 节流：两个原因叠加

利用率：稳态 2.175 ÷ 3 = 73%，首 10 秒 2.55 ÷ 3 = 85%，已越过 0.7 的容量线，排队倍数约 3.6 与 6.7（第 8 页）

节流：12 个忙线程，300 ms 配额 25 ms 用完，每周期被停 75 ms（第 12 页）；38% 的周期被节流

改法：limit ≥ 2.55 ÷ 0.7 = 3.64，取 4 核；或去掉硬 limit 只留 request（代价：失控的进程可占满空闲核）；或把并发线程数限制在 limit 核数以内（代价：排队移到池外）

常见误答：「批处理抢了 CPU」。节点 CPU 只有 67%，批处理平时 1 核；节流计数指向的是 Tomcat 自己的阀门

2 被杀：超出的是 cgroup limit

被杀的原因是超过了 cgroup 的 limit，而不是整机内存不够：5.1 > 5.0

limit = 观测峰值 5.1 GB + 约 8% 余量 ≈ 5.5 GB，构成见第 14 页；request 取同值，因为内存不可压缩

不能按平时的 4.5 GB 定，也不能只看节点还剩多少

结论 变慢与被杀都出自容器自己的阀门：先读 cgroup 的计数器，再看节点总量

讨论要点（3） （4）

3 批处理：Burstable

很小的保证加明确的上限：request 0.5 核、0.5 GB，limit 4 核、3 GB。用量远超 request，内存紧张时最先被驱逐；理由是可重跑、延迟不敏感

不用 BestEffort：它没有 limit，失控时无界占用

若给 Guaranteed（request = limit = 4 核）：Σrequest = 5.2 + 4 = 9.2 > 8，节点 1 放不下

代价由使用者承担：完成时间变长（L-08 第 12 页）；被终止后重跑会浪费已做的工作，需要检查点；须让教务统计一方事先知道

4 超卖：成立，靠优先级兜底

首 10 秒节点 1 的需求：2 × 2.55 + 4 = 9.1 > 8。按 request 加权分配，Tomcat 拿到全部 5.1 核，批处理被压到 2.9 核

依据不是错峰（批处理恰好与开放日重叠），而是优先级兜底。条件有三：资源可压缩；被压的一方可降级；Σrequest 5.7 ≤ 8

追问：内存也能这样超卖吗？不能。若批处理内存 limit 设 8 GB，Σlimit 19 GB > 16 GB；实际 2 × 5.1 + 6 = 16.2 GB，节点级 OOM，必有一个被杀。所以内存不超卖（14 ÷ 16 = 0.875）

结论 开放日的 CPU 超卖靠优先级兜底才成立，代价由批处理承担；内存不超卖

参考配额表

负载	CPU request	CPU limit	内存 request	内存 limit	QoS	理由
Tomcat × 4	2.6	4	5.5 GB	5.5 GB	Burstable（内存 request = limit）	峰值 2.55；÷ 0.7 取 4；5.1 加约 8%
批处理	0.5	4	0.5 GB	3 GB	Burstable	可重跑，最先让
MySQL	—	—	—	—	不在池内	独占 4 核 8 GB；有状态、fsync 敏感

节点	资源	容量	Σrequest	Σlimit	limit ÷ 容量
节点 1	CPU	8	5.7	12	**1.5**
节点 1	内存	16 GB	11.5	14	0.875
节点 2	CPU	8	5.2	8	1.0
节点 2	内存	16 GB	11	11	0.69

入口限流（读）

16 核 × 0.7 = 11.2 核；扣除写的占用（首 10 秒 600 × 5 ms = 3.0 核）：(11.2 − 3.0) ÷ 3 ms ≈ 2730 QPS

稳态写 300 TPS 时约 3230 QPS；2400 ÷ 2730 ≈ 88%，首 10 秒余量约 12%

写的阈值由磁盘决定（第 22 页）

结论 CPU 可以超卖，靠 request 加权与优先级；内存不超卖；有状态服务独占

本讲小结

- **三问加一个保证**
  - 给多少、放哪儿、何时变；隔离保证给的就是拿到的
- **可压缩的超出时变慢，不可压缩的超出时失败**
  - 节流看周期内的并发；内存 limit 按峰值加余量定
- **request 管不被挤，limit 管不拖累别人**
  - 超卖靠错峰或优先级兜底；内存不超卖；QoS 写明出事时谁先让
- **请求级阀门管放多少进来**
  - 限流、并发上限、有界队列、优先级丢弃、舱壁

判断标准 这份资源给谁、给多少？不够或超了，谁受损、代价是什么？

课后思考与参考资料

课后思考

1. 为小组项目的各组件写一张配额表：request、limit、QoS 级别，并算出每个节点的 Σrequest、Σlimit 和超卖比。

2. limit 1.5 核、6 个线程同时忙，每个周期被停多久？此时容器的平均 CPU 用量显示为多少？

3. 把 JVM 的 -Xmx 从 4 GB 调到 5 GB，容器内存 limit 仍是 5.5 GB，开放日会发生什么？为什么？

参考资料

Linux kernel docs：Control Group v2；CFS Bandwidth Control

Kubernetes docs：Resource Management for Pods and Containers；Pod Quality of Service Classes；Node-pressure Eviction

B. Gregg, Systems Performance, 2nd ed.：Ch. 6 CPUs、Ch. 7 Memory、Ch. 11 Cloud Computing

Google SRE Book：Ch. 21 Handling Overload

OpenJDK JEP 346；Go 1.25 Release Notes（容器感知的 GOMAXPROCS）

感谢聆听

饮 水 思 源　爱 国 荣 校